RevoGrocers Sales Performance Analysis

Muhammad Maulana

COMPANY OVERVIEW:

RevoGrocers is a fictional retail grocery chain with multiple store locations selling diverse grocery products.

The company leverages data insights to refine **sales tactics, improve customer satisfaction, and increase overall revenue.**

Disclaimer

- available Kaggle dataset and does not reflect real-world financial or business insight
- RevoGrocers is a fictional entity, and the results presented here are purely for educational purposes
- The queries and insight generated in this assignment is personalized to My own analysis results

Dataset overview

Key Table Used

- sales (transactional sales data)
- customers (customer information)
- products (product details)
- categories (product category information)

Relevant Tables Used

- Employees (sales representative)
- Cities(customer city)
- Countries (customer country)

DATASET SOURCE

This analysis is based on a public dataset from Kaggle: (CLICK HERE)

PROJECT GOAL

1. Identify the product category that generates the highest revenue after discount.
2. Assess the relation between revenue after discount and total units sold for each product category.
3. Find the relation between revenue after discount and the number of unique customers for each product category.
4. Calculate the average price unit for each product category in catalogue.
5. Evaluate the relation between the average price per unit and the number of buyers (unique customers) per category.
6. Which categories contribute the most to overall revenue after discount (percentage-wise)?
7. Which product categories have the highest repeat purchase rate?
8. After executing the SQL queries, summarize your overall findings regarding the sales performance across product categories.
9. Find the cumulative amount of transaction of the top user (user with highest transaction value, window function)

Methodology

Dataset	• Reviewed the dataset structure and table relationship and identified the relevant table
Data Preparation	• Joined the related table, filtered and selected required records based on Key problem
SQL Queries	• Used JOINs,CTEs,aggregate functions , and window functions to answer key problem
Analysis &insights	• Interpreted the result to identify trends, evaluate category performance for insights

Key Problem:
Identify the highest-performing product category based on revenue after discounts.

```
QUERY:

SELECT
  cat.categoryname as category_name,
  SUM(prod.price *sales.Quantity * (1 -
sales.Discount)) AS total_revenue
FROM fsda-sql-01.grocery_dataset.categories AS
cat
JOIN fsda-sql-01.grocery_dataset.products AS prod
ON cat.categoryid = prod.categoryid
JOIN fsda-sql-01.grocery_dataset.sales AS sales
ON sales.ProductID = prod.productid
GROUP BY 1
ORDER BY 2 DESC
LIMIT 3
```

Row	category_name	total_revenue
1	Confections	556930717.3468...
2	Meat	492888844.6946...
3	Poultry	440025564.9656...

The Confections category achieved the highest revenue after discounts, with Meat and Poultry ranking second and third, respectively. These categories represent the business's top revenue drivers.

Key Problem:Assess the relation between revenue after discount and total units sold for each product category.

```
QUERY:
SELECT
  cat.categoryname as category_name,
  SUM(sales.Quantity) AS total_unit_sold,
  SUM(prod.price *sales.Quantity * (1 -
sales.Discount)) AS total_revenue
FROM fsda-sql-01.grocery_dataset.categories AS
cat
JOIN fsda-sql-01.grocery_dataset.products AS prod
ON cat.categoryid = prod.categoryid
JOIN fsda-sql-01.grocery_dataset.sales AS sales
ON sales.ProductID = prod.productid
GROUP BY 1
ORDER BY 3 DESC
LIMIT 5
```

Row	category_name	total_unit_sold	total_revenue
1	Confections	11078474	556930717.3468...
2	Meat	9719292	492888844.6946...
3	Poultry	9159847	440025564.9656...
4	Cereals	8735296	427393431.9101...
5	Snails	7199409	372084885.2075...

Revenue is entirely volume-driven: Sales volume (total_unit_sold) directly dictates total revenue, as the highest-volume category (Confections) yields the highest revenue and the lowest-volume category (Snails) yields the lowest.

Key Problem:The relation between revenue after discount and the number of unique customers for each product category.

```
QUERY:
SELECT
  cat.categoryname as category_name,
  COUNT(DISTINCT sales.CustomerID) AS
total_unique_cust,
  SUM(prod.price *sales.Quantity * (1 -
sales.Discount)) AS total_revenue

FROM fsda-sql-01.grocery_dataset.categories AS cat
JOIN fsda-sql-01.grocery_dataset.products AS prod
ON cat.categoryid = prod.categoryid
JOIN fsda-sql-01.grocery_dataset.sales AS sales
ON sales.ProductID = prod.productid
GROUP BY 1
ORDER BY 3 DESC
LIMIT 5
```

Row	category_name	total_unique_cust	total_revenue
1	Confections	98743	556930717.3468...
2	Meat	98701	492888844.6946...
3	Poultry	98679	440025564.9656...
4	Cereals	98651	427393431.9101...
5	Snails	98376	372084885.2075...

Customer reach is nearly identical across categories, spanning 98,376 to 98,743 unique buyers (<0.4% difference).

Key Problem:Average price unit for each product category

```
QUERY:

SELECT
  cat.categoryname as category_name,
  AVG(prod.price) as average_price_unit

FROM fsda-sql-01.grocery_dataset.categories AS cat
JOIN fsda-sql-01.grocery_dataset.products AS prod
ON cat.categoryid = prod.categoryid

GROUP BY 1
ORDER BY 2 DESC
LIMIT 5
```

Row	category_name	average_price_unit
1	Grain	61.40394642857...
2	Dairy	53.55683142857...
3	Snails	53.19933243243...
4	Meat	52.27465199999...
5	Confections	51.84993157894...

Grain commands a distinct premium price: Grain leads all product categories with the highest average unit price at ~$61.40, standing significantly higher than the rest of the catalogue.

Key Problem: Evaluate the relation between the average price per unit and the number of buyers (unique customers) per category.

```
QUERY:
with avg_price
 AS(
  SELECT
  cat.categoryname as category_name,
  AVG(prod.price) as average_price_unit
  FROM fsda-sql-01.grocery_dataset.categories AS cat
  JOIN fsda-sql-01.grocery_dataset.products AS prod
  ON cat.categoryid = prod.categoryid
  GROUP BY 1
)
,
uniq_cust AS(
  SELECT
  cat.categoryname as category_name,
  Count(DISTINCT sales.CustomerID) AS total_uniq_cust
    FROM fsda-sql-01.grocery_dataset.categories AS cat
    JOIN fsda-sql-01.grocery_dataset.products AS prod
    ON cat.categoryid = prod.categoryid
    JOIN fsda-sql-01.grocery_dataset.sales AS sales
    ON sales.ProductID = prod.productid
    GROUP BY 1
)

SELECT
avg_price.category_name,
avg_price.average_price_unit,
uniq_cust.total_uniq_cust

FROM avg_price
JOIN uniq_cust
ON avg_price.category_name = uniq_cust.category_name

LIMIT 5
```

Key Problem: Evaluate the relation between the average price per unit and the number of buyers (unique customers) per category.

Row	category_name	average_price_unit	total_uniq_cust
1	Confections	51.84993157894...	98743
2	Shell fish	44.27278333333...	98338
3	Cereals	50.41638888888...	98651
4	Dairy	53.55683142857...	98308
5	Beverages	51.04368421052...	98424

Price elasticity has no impact on customer acquisition: Customer volume (total_uniq_cust) remains exceptionally steady across all categories (~98,300 to ~98,700 buyers), showing that higher unit prices (like Dairy at ~$53.56) do not deter buyers compared to lower-priced items (like Shell fish at ~$44.27).

Key Problem: Which categories contribute the most to overall revenue after discount (percentage-wise)

```
QUERY:
WITH category_revenue AS(
  SELECT
  cat.categoryname as category_name,
  SUM(prod.price *sales.Quantity * (1 -
sales.Discount)) AS total_revenue

FROM fsda-sql-01.grocery_dataset.categories AS
cat
JOIN fsda-sql-01.grocery_dataset.products AS
prod
ON cat.categoryid = prod.categoryid
JOIN fsda-sql-01.grocery_dataset.sales AS sales
ON sales.ProductID = prod.productid
GROUP BY 1

)
```

```
SELECT
  category_name,
  total_revenue,
  SUM(total_revenue)OVER () AS sum_total_rev,
  ROUND(total_revenue/SUM(total_revenue)OVER
()*100,1) AS rev_percentage

FROM category_revenue
ORDER BY 4
```

Key Problem: Which categories contribute the most to overall revenue after discount (percentage-wise)

Row	category_name	total_revenue	sum_total_rev	rev_percentage
1	Shell fish	299598294.0299...	4327063166.563...	6.9
2	Grain	323879126.6500...	4327063166.563...	7.5
3	Seafood	330527987.4676...	4327063166.563...	7.6
4	Dairy	354358156.7688...	4327063166.563...	8.2
5	Produce	362861133.5171...	4327063166.563...	8.4
6	Beverages	366515024.0050...	4327063166.563...	8.5
7	Snails	372084885.2075...	4327063166.563...	8.6
8	Cereals	427393431.9101...	4327063166.563...	9.9
9	Poultry	440025564.9656...	4327063166.563...	10.2
10	Meat	492888844.6946...	4327063166.563...	11.4
11	Confections	556930717.3468...	4327063166.563...	12.9

Top 3 categories drive over one-third of total revenue: Confections (12.9%), Meat (11.4%), and Poultry (10.2%) combined account for 34.5% of the entire revenue pool (~4.33 Billion).

Key Problem: Which product categories have the highest repeat purchase rate?

```sql
WITH cust_purchase AS(
  SELECT
    cat.categoryname AS cat_name,
    sales.CustomerID AS cust_id,
    COUNT(DISTINCT sales.SalesID) AS total_purchase

  FROM fsda-sql-01.grocery_dataset.categories AS cat
  JOIN fsda-sql-01.grocery_dataset.products AS prod
  ON cat.categoryid = prod.categoryid
  JOIN fsda-sql-01.grocery_dataset.sales AS sales
  ON sales.ProductID = prod.productid
  GROUP BY 1,2
)
,

repeat_cust AS(
  SELECT
  cat.categoryname AS cat_name,
    sales.CustomerID AS cust_id,
    COUNT(DISTINCT sales.SalesID) AS total_purchase

  FROM fsda-sql-01.grocery_dataset.categories AS cat
  JOIN fsda-sql-01.grocery_dataset.products AS prod
  ON cat.categoryid = prod.categoryid
  JOIN fsda-sql-01.grocery_dataset.sales AS sales
  ON sales.ProductID = prod.productid
  GROUP BY 1,2
  HAVING COUNT(DISTINCT sales.SalesID) >1
)
,
CTE1 AS(
  SELECT
    cat_name,
    COUNT(DISTINCT cust_id) AS tot_cust
  FROM cust_purchase
  GROUP BY 1

)
,
CTE2 AS(
  SELECT
  cat_name,
  COUNT(DISTINCT cust_id) AS tot_cust_repeat
  FROM repeat_cust
  GROUP BY 1
)
```

Final query

```sql
SELECT
  CTE1.cat_name,
  CTE1.tot_cust,
  CTE2.tot_cust_repeat,
  ROUND(tot_cust_repeat / tot_cust * 100,2) AS
cust_repeat_pct
FROM CTE1
JOIN CTE2
ON CTE1.cat_name = CTE2.cat_name
ORDER BY 4 DESC
```

Key Problem: Which product categories have the highest repeat purchase rate?

Row	cat_name	tot_cust	tot_cust_repeat	cust_repeat_pct
1	Confections	98743	98598	99.85
2	Meat	98701	98318	99.61
3	Poultry	98679	98122	99.44
4	Cereals	98651	97867	99.21
5	Produce	98601	97550	98.93
6	Beverages	98424	96679	98.23
7	Snails	98376	96324	97.91

Confections boasts both the highest customer count (98,743) and the top repeat purchase rate (99.85%), making it the primary driver for customer retention.

Observation	Business Impact	Isolation	Prioritization	Reccomendation
The Confections category achieved the highest revenue after discounts, with Meat and Poultry ranking second and third, respectively. These categories represent the business's top revenue drivers.	These categories contribute the largest share of company revenue, making them critical to overall sales performance.	The result is driven by high customer demand and/or larger transaction values within these categories.	High	Prioritize inventory availability, marketing campaigns, and promotional budgets for these top-performing categories to maximize revenue.
Revenue is entirely volume-driven: Sales volume (total_unit_sold) directly dictates total revenue, as the highest-volume category (Confections) yields the highest revenue and the lowest-volume category (Snails) yields the lowest.	Increasing sales volume has a direct impact on revenue growth. Low-volume categories contribute less to total sales.	Compare pricing and customer demand between high- and low-volume categories to determine whether low revenue is due to low demand or pricing.	Medium	Focus promotions and cross-selling strategies to increase sales volume in underperforming categories while maintaining demand for high-volume products.
Customer reach is nearly identical across categories, spanning 98,376 to 98,743 unique buyers (<0.4% difference).	Customer exposure is evenly distributed, suggesting revenue differences are driven more by purchasing behavior than customer reach.	Analyze purchase frequency, average order value, and repeat purchase rate by category to identify what drives higher revenue.	Medium	Increase repeat purchases and basket size through loyalty programs, targeted promotions, and product bundling instead of focusing only on acquiring new customers.
Grain has the highest average unit price (~$61.40), positioning it as the premium product category.	Premium pricing can increase revenue and profit per sale.	Determine whether the higher price is supported by customer demand and purchase frequency.	Medium	Maintain premium pricing if demand remains stable, while promoting Grain's value through quality-focused marketing and targeted offers.

Observation	Business Impact	Isolation	Prioritization	Reccomendation
Customer acquisition remains consistent across categories despite differences in average unit price, suggesting price has minimal impact on attracting customers.	Customer acquisition is relatively insensitive to product price, indicating that price differences are not the primary factor influencing category participation.	Since customer reach is stable across categories, investigate purchase frequency, basket size, and product assortment as the main drivers of revenue differences.	Low	Rather than lowering prices to attract more customers, focus on increasing purchase frequency and average order value through bundles, loyalty incentives, and targeted promotions.
Confections has both the highest unique customer count (98,743) and the highest repeat purchase rate (99.85%), making it the strongest contributor to customer retention.	High repeat purchasing creates stable and recurring revenue, reducing reliance on acquiring new customers.	Compare product assortment, pricing strategy, and promotional activities in Confections with other categories to identify practices that encourage repeat purchases.	High	Expand successful Confections strategies to other categories through loyalty rewards, personalized offers, and cross-category promotions to improve customer retention.
The top three categories—Confections (12.9%), Meat (11.4%), and Poultry (10.2%)—generate 34.5% of total revenue (~4.33 billion), making them the company's primary revenue contributors.	A significant portion of revenue depends on a small number of categories, making their performance critical to overall business success.	Monitor sales trends, inventory availability, and promotional effectiveness for these categories, as performance declines would have a disproportionate impact on total revenue.	High	Prioritize inventory management, marketing investment, and demand forecasting for these categories while gradually diversifying revenue through growth initiatives in lower-performing categories.

Key Problem : Find the cumulative amount of transaction of the top user (user with highest transaction value, window function)

```sql
WITH cte1 AS(
  SELECT
    CustomerID,
    SUM(price * Quantity *(1-Discount)) AS total_rev

  FROM fsda-sql-01.grocery_dataset.sales AS sales
  LEFT JOIN fsda-sql-01.grocery_dataset.products AS prod
  ON sales.productID = prod.productid
  GROUP BY 1
)
,

cte2 AS(
  SELECT
    CustomerID
  FROM cte1
  ORDER BY total_rev DESC
  LIMIT 1
)
```

```sql
SELECT
 cte2.CustomerID,
 cust.firstname,
 cust.lastname,
 TransactionNumber,
 SalesDate,
 prod.price * sales.Quantity * (1 - sales.Discount) AS transaction_value,
 SUM(prod.price * sales.Quantity * (1 - sales.Discount))
OVER (
    ORDER BY SalesDate,TransactionNumber
) AS cumulative_amount


FROM cte2
JOIN fsda-sql-01.grocery_dataset.sales AS sales
ON cte2.CustomerID = sales.CustomerID
JOIN fsda-sql-01.grocery_dataset.customers AS cust
ON cte2.CustomerID = cust.customerid
JOIN fsda-sql-01.grocery_dataset.products AS prod
ON sales.productID = prod.productid
```

Key Problem : Find the cumulative amount of transaction of the top user (user with highest transaction value, window function)

Row	CustomerID	firstname	lastname	TransactionNumber	SalesDate	transaction_amo...	cumulative_amou...
1	94800	Wayne	Chan	08G12Y3UIQ2OB9MUELK9	*null*	2013.004800000...	3738.26496
2	94800	Wayne	Chan	5D2SZPPUS6AZ31ECU24O	*null*	1725.260159999...	3738.26496
3	94800	Wayne	Chan	AK6B7A7YG3L9LDD370RY	2018-01-01 02:19:11.270000 UTC	2366.3472	6104.612160000...
4	94800	Wayne	Chan	3RCDALJW0RIR5AU0CK02	2018-01-04 03:16:26.030000 UTC	2251.312799999...	8355.92496
5	94800	Wayne	Chan	B826FGU5WFR2ESCG2OSB	2018-01-04 15:06:45.760000 UTC	342.4512000000...	8698.37616
6	94800	Wayne	Chan	8BROO5KFVQ4Y3SDOFOLI	2018-01-05 01:40:34.040000 UTC	377.3592	9075.73536
7	94800	Wayne	Chan	YEJZWXXOK3U4CSEYZC6O	2018-01-07 17:01:02.900000 UTC	1166.354400000...	10242.08976
8	94800	Wayne	Chan	JB1D6I71SIIIXRU70W08E	2018-01-08 15:14:29.770000 UTC	2245.4712	12487.56096
9	94800	Wayne	Chan	TDAURMBXWHKFB3MWDMEK	2018-01-08 17:54:00.690000 UTC	1501.104	13988.66496
10	94800	Wayne	Chan	8HROSTUPFNPCNBEO9O04	2018-01-09 18:38:29.950000 UTC	1493.91936	15482.58432
11	94800	Wayne	Chan	3Z1NBJF5PB02N0XUL268	2018-01-10 17:58:44.360000 UTC	1111.804800000...	16594.38912
12	94800	Wayne	Chan	B7RCNJYBVBINR0BMB7XA	2018-01-11 05:17:41.040000 UTC	2313.904800000...	18908.29392
13	94800	Wayne	Chan	XCSM2KF7LFDV79C4R2OO	2018-01-11 06:41:40.790000 UTC	1524.933599999...	20433.22752
14	94800	Wayne	Chan	YN7KS0R8VQDCOYN4AT26	2018-01-17 09:32:05.180000 UTC	1038.8736	21472.10112
15	94800	Wayne	Chan	BEZOGCH60HMAXEBUAYGS	2018-01-19 03:01:19.370000 UTC	525.1344	21997.23552000...
16	94800	Wayne	Chan	A6XSVFK48S2MM3LUCV65	2018-01-19 08:52:07 UTC	1113.6792	23110.91472
17	94800	Wayne	Chan	GY5LGRCQ767N5DL4FU0M	2018-01-19 15:36:47.010000 UTC	843.22944	23954.14416
18	94800	Wayne	Chan	8FHH962BP4ELMTBZFLTU	2018-01-20 06:53:53.360000 UTC	585.3912	24539.53536
19	94800	Wayne	Chan	MHE9HRULRFUH0RZRI1ZC	2018-01-24 09:20:35.320000 UTC	1303.6608	25843.19616
20	94800	Wayne	Chan	S3SKJ7CXN0PPAUFF1YE8	2018-01-25 12:51:51.010000 UTC	482.4216	26325.61776
21	94800	Wayne	Chan	D0M2UC92BHBUT0AR066Q	2018-01-26 23:02:16.870000 UTC	2313.2904	28638.90816
22	94800	Wayne	Chan	LCML8NDNRXQ9B3B3B44R	2018-01-27 22:55:58.600000 UTC	687.6792	29326.58736
23	94800	Wayne	Chan	F3Y8EJU8FLOW6TES6LFZ	2018-01-28 16:28:26.740000 UTC	2162.7768	31489.36416
24	94800	Wayne	Chan	RYWI4AK1416EWXZMOU9R	2018-01-30 16:09:10.120000 UTC	2357.9304	33847.29456
25	94800	Wayne	Chan	HBP5QBLMPNOCHAQ5XS7X	2018-01-30 22:29:59.670000 UTC	512.7552	34360.04976

Key Problem : Find the cumulative amount of transaction of the top user (user with highest transaction value, window function)

Row	CustomerID	firstname	lastname	TransactionNumber	SalesDate	transaction_amo...	cumulative_amou...
24	94800	Wayne	Chan	RYWH4AK1416EWAZMOU9R	2018-01-30 16:09:10.120000 UTC	2357.9304	33847.29456
25	94800	Wayne	Chan	HBP5QBLMPNOCHAQ5XS7X	2018-01-30 22:29:59.670000 UTC	512.7552	34360.04976
26	94800	Wayne	Chan	FIUK22R5LGPX98A8OHUK	2018-01-31 08:26:37.770000 UTC	411.0576000000...	34771.10736
27	94800	Wayne	Chan	1IQ7TF4OAANFYOV10GWR	2018-02-01 19:43:58.130000 UTC	2109.6552	36880.76256
28	94800	Wayne	Chan	ALRLGB7NM5E36E8USW3H	2018-02-05 16:35:29.860000 UTC	1.077600000000...	36881.84016
29	94800	Wayne	Chan	XM9N9KJDKHNTR6GUIZZ9	2018-02-06 16:04:08.650000 UTC	2296.8576	39178.69776
30	94800	Wayne	Chan	ZOSX7YBU220GQC7896YL	2018-02-08 22:43:51.410000 UTC	1829.759999999...	41008.45776
31	94800	Wayne	Chan	YGP374GPU56P56D05MJ4	2018-02-10 08:52:33.160000 UTC	1152.64944	42161.1072
32	94800	Wayne	Chan	8MWD8CQIKAJFN3QXMBTG	2018-02-10 13:56:23.030000 UTC	994.0876800000...	43155.19488
33	94800	Wayne	Chan	J9YQS47HC00BRYGK3MBG	2018-02-11 20:49:09.190000 UTC	1334.5416	44489.73648
34	94800	Wayne	Chan	00QSBPLEH6Y32SP4XWJS	2018-02-12 08:39:17.560000 UTC	1307.1432	45796.87968
35	94800	Wayne	Chan	EIG23T8SDGWK4C3BIF9U	2018-02-12 17:54:11.600000 UTC	1845.571199999...	47642.45088
36	94800	Wayne	Chan	8D0I8TMT0IN77QUKMMUE	2018-02-12 23:11:18.700000 UTC	163.9968	47806.44768
37	94800	Wayne	Chan	G8QBZZG5Z9130PNS2IR7	2018-02-13 13:20:07.520000 UTC	1866.729600000...	49673.17728
38	94800	Wayne	Chan	VMNMPHPTE9Q7Q7HS3JAI	2018-02-15 15:49:37.860000 UTC	1342.392960000...	51015.57024
39	94800	Wayne	Chan	VX2RQ1577DRV0XEABRY2	2018-02-16 05:52:18.980000 UTC	1328.8392	52344.40944
40	94800	Wayne	Chan	Z6D0W4SOAI6GG7ORFJFY	2018-02-20 14:44:15.520000 UTC	1093.3224	53437.73184
41	94800	Wayne	Chan	S4IJ94J7KYTVD5WMTXWC	2018-02-22 04:04:47.120000 UTC	2109.6552	55547.38704
42	94800	Wayne	Chan	3GCX6X8WMTGOEMFKXEDB	2018-02-25 17:59:13.110000 UTC	369.9518400000...	55917.33888
43	94800	Wayne	Chan	FFOSN3Q7CKVXJ8DZ4JO0	2018-02-26 14:22:58.150000 UTC	1963.0632	57880.40208
44	94800	Wayne	Chan	4F6USP6SOCDI12I7WLIR	2018-02-27 00:15:18.240000 UTC	57.336	57937.73808
45	94800	Wayne	Chan	CCZF15VJEA3NB7JF41G7	2018-02-27 22:23:52.060000 UTC	146.3424	58084.08048000...
46	94800	Wayne	Chan	2QDVTI9BWHERFN3HR5MS	2018-03-02 14:14:33.640000 UTC	1307.1432	59391.22368
47	94800	Wayne	Chan	KAYG1932DTQR5Q9RXHLJ	2018-03-02 17:11:15.480000 UTC	449.64504	59840.86872
48	94800	Wayne	Chan	L5CR0154JIY6IEJ4E1U7	2018-03-03 15:24:26.620000 UTC	1703.408400000...	61544.27712
49	94800	Wayne	Chan	Z4PZZFL7ETE3NCBR7T5M	2018-03-06 12:48:41.020000 UTC	1670.1144	63214.39152
50	94800	Wayne	Chan	IJI873KMU66D03IXBUF9	2018-03-07 16:45:57.070000 UTC	182.4455999999...	63396.83712000...